Perceptron Learning Algorithm and Its Limitations

Haithem Bekkari

February 15, 2026

1 Introduction

This laboratory work explores foundational neural network models, including the McCulloch–Pitts neuron and Rosenblatt's perceptron. The perceptron learning algorithm is implemented and evaluated through visualization of its decision boundary, convergence behavior, and sensitivity to the learning rate. Additionally, its limitation on non-linearly separable problems such as XOR is analyzed.

2 Implementation

The perceptron computes a weighted linear combination of inputs followed by a step activation function:

$$y = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

The weights are updated using the perceptron learning rule:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}, \quad b \leftarrow b + \eta(y - \hat{y})$$

where η represents the learning rate.

All experiments were implemented in Python using NumPy, Matplotlib, and Scikit-learn for synthetic data generation.

3 Results

3.1 Decision Boundary and Convergence

Figure 1 illustrates the perceptron's learned decision boundary alongside its learning curve. The left panel shows the linear separation between the two classes, while the right panel displays the number of misclassifications per epoch. The rapid decrease in error demonstrates convergence of the algorithm.

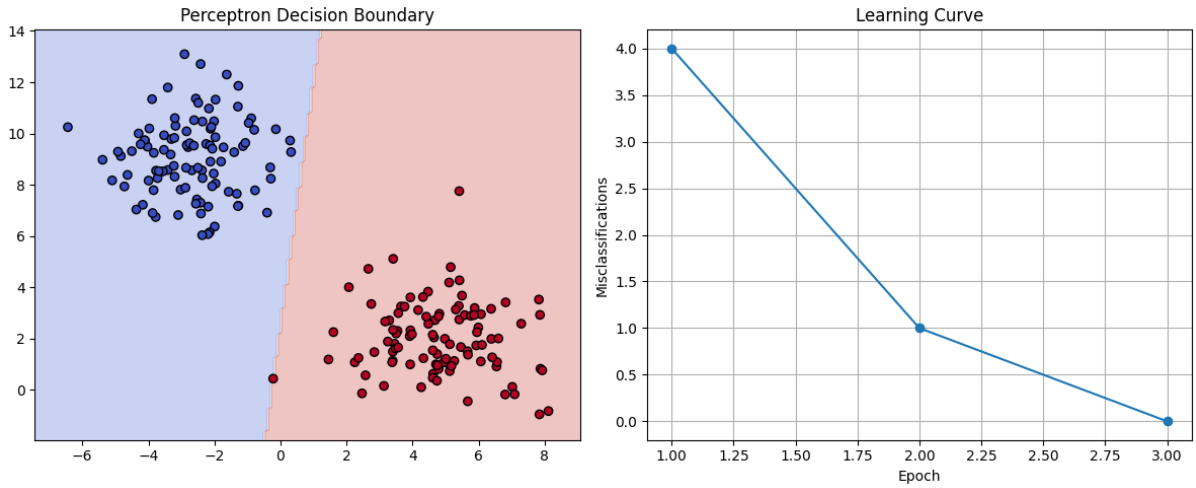

Figure 1: Perceptron decision boundary (left) and learning curve (right).

3.2 Learning Rate Sensitivity

To evaluate the influence of the learning rate, multiple models were trained using different η values. Figure 2 compares their convergence behavior.

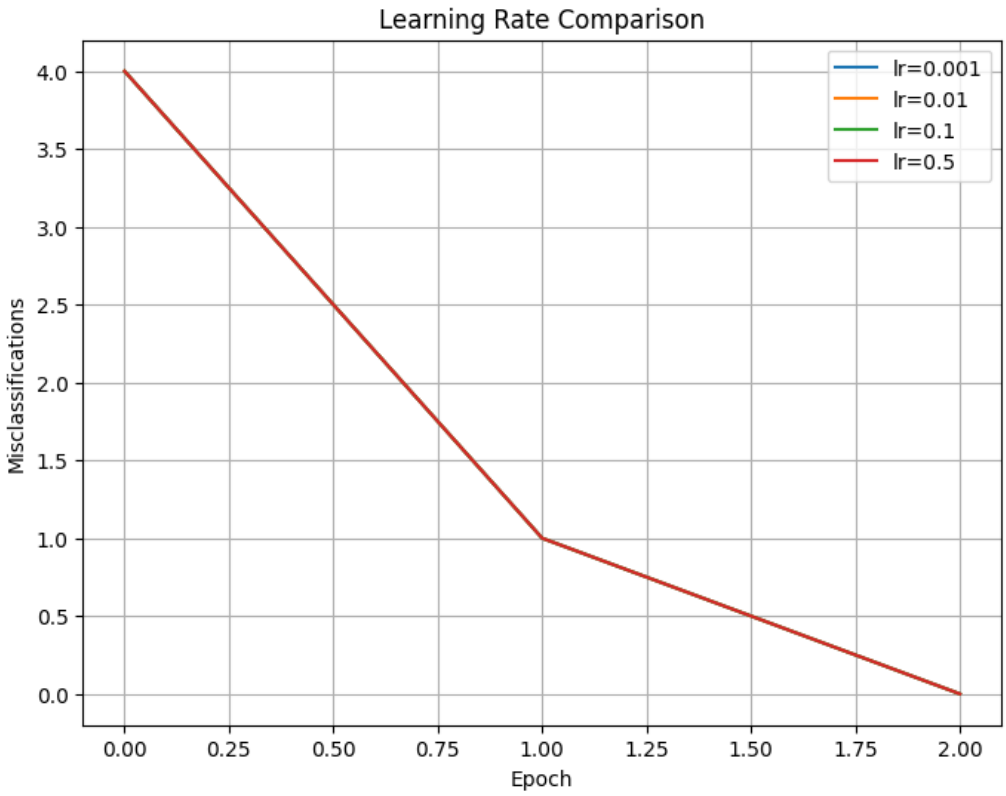

Figure 2: Comparison of convergence behavior for different learning rates.

All tested learning rates converged rapidly on this linearly separable dataset. Due to the simplicity of the data, their convergence curves overlap, making their behavior visually similar.

4 Discussion

Why the Perceptron Fails on XOR

A single-layer perceptron can only represent linearly separable functions. The XOR problem is not linearly separable; therefore, no linear decision boundary can correctly classify all input points. Solving XOR requires introducing hidden layers, as in multi-layer perceptrons.

Effect of the Learning Rate

The learning rate controls the step size of weight updates:

- Small $\eta \rightarrow$ slow convergence
- Large $\eta \rightarrow$ potential oscillation
- Moderate $\eta \rightarrow$ efficient and stable learning

McCulloch–Pitts vs. Perceptron

The McCulloch–Pitts neuron operates with fixed weights and does not learn from data. The perceptron introduced adaptive weight updates, marking a fundamental advancement toward modern neural networks.

5 Conclusion

The perceptron successfully learns linearly separable datasets and converges efficiently under appropriate hyperparameters. However, its inability to model non-linear decision boundaries, such as XOR, highlights the necessity of deeper neural architectures. This limitation ultimately motivated the development of multi-layer neural networks.